

TP : Ajout d'un API Gateway dans une application microservice

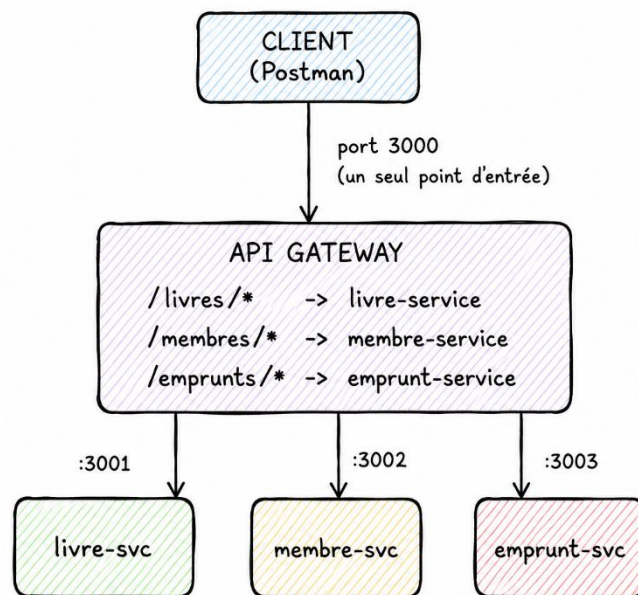
Objectif

Ajouter un API Gateway devant les 3 microservices existants. Le gateway sera le seul point d'entree pour le client — un seul port 3000 qui redirige vers le bon service. Les 3 services existants ne sont pas modifies.

1. Principe du Gateway

Dans une architecture microservices, chaque service tourne sur son propre port et expose ses propres routes. Sans gateway, le client doit connaitre l'adresse et le port de chaque service, c'est ingerable en production et dangereux car on expose tous les ports directement.

L'API Gateway est un serveur intermediaire qui joue le role de point d'entree unique : le client envoie toutes ses requetes sur un seul port, le gateway analyse l'URL, identifie le service responsable et transfere la requete vers lui via axios. Le client ne sait pas combien de services existent derriere, ni sur quels ports ils tournent, il ne voit que le gateway. C'est aussi l'endroit ideal pour centraliser des fonctionnalites transversales comme l'authentification, la gestion des erreurs ou le logging.



3. Etape 1 — Creer le dossier gateway

On cree un nouveau dossier gateway a cote des 3 services existants. On n'y touche pas.

Structure finale du projet :

bibliotheque-microservices/

livre-service/ <- inchange

membre-service/ <- inchange

emprunt-service/ <- inchange

gateway/ <- nouveau

index.js

package.json

Commandes :

```
mkdir gateway
cd gateway
npm init -y
npm install express axios mongodb
```

4. Etape 2 — Créer gateway/index.js

Ce fichier contient le serveur Express du gateway. Pour chaque route, il appelle le service correspondant avec axios et retourne la réponse au client.

Initialisation :

```
const express = require('express')
const axios = require('axios')

const app = express()
app.use(express.json())

const LIVRE_SERVICE = 'http://localhost:3001'
const MEMBRE_SERVICE = 'http://localhost:3002'
const EMPRUNT_SERVICE = 'http://localhost:3003'
```

Route livres :

```
app.get('/livres', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres`)
    res.status(200).json(r.data)
  } catch (err) { res.status(500).json({ message: err.message }) }
})

app.get('/livres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres/${req.params.id}`)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ message:
'Livre non trouve' })
    res.status(500).json({ message: err.message })
  }
})

app.post('/livres', async (req, res) => {
  try {
    const r = await axios.post(`${LIVRE_SERVICE}/livres`, req.body)
    res.status(201).json(r.data)
  } catch (err) { res.status(500).json({ message: err.message }) }
})
```

```

}))

app.put('/livres/:id', async (req, res) => {
  try {
    const r = await axios.put(`${LIVRE_SERVICE}/livres/${req.params.id}`,
    req.body)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ message:
    'Livre non trouve' })
    res.status(500).json({ message: err.message })
  }
})

app.delete('/livres/:id', async (req, res) => {
  try {
    const r = await axios.delete(`${LIVRE_SERVICE}/livres/${req.params.id}`)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ message:
    'Livre non trouve' })
    res.status(500).json({ message: err.message })
  }
})

```

Routes membres :

```

app.get('/membres', async (req, res) => {
  try {
    const r = await axios.get(`${MEMBRE_SERVICE}/membres`)
    res.status(200).json(r.data)
  } catch (err) { res.status(500).json({ message: err.message }) }
})

app.get('/membres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${MEMBRE_SERVICE}/membres/${req.params.id}`)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ message:
    'Membre non trouve' })
    res.status(500).json({ message: err.message })
  }
})

app.post('/membres', async (req, res) => {
  try {
    const r = await axios.post(`${MEMBRE_SERVICE}/membres`, req.body)

```

```

        res.status(201).json(r.data)
    } catch (err) { res.status(500).json({ message: err.message }) }
})

app.put('/membres/:id', async (req, res) => {
    try {
        const r = await axios.put(`${MEMBRE_SERVICE}/membres/${req.params.id}`,
req.body)
        res.status(200).json(r.data)
    } catch (err) {
        if (err.response?.status === 404) return res.status(404).json({ message:
'Membre non trouve' })
        res.status(500).json({ message: err.message })
    }
})

app.delete('/membres/:id', async (req, res) => {
    try {
        const r = await axios.delete(`${MEMBRE_SERVICE}/membres/${req.params.id}`)
        res.status(200).json(r.data)
    } catch (err) {
        if (err.response?.status === 404) return res.status(404).json({ message:
'Membre non trouve' })
        res.status(500).json({ message: err.message })
    }
})

```

Routes emprunts :

```

app.get('/emprunts', async (req, res) => {
    try {
        const r = await axios.get(`${EMPRUNT_SERVICE}/emprunts`)
        res.status(200).json(r.data)
    } catch (err) { res.status(500).json({ message: err.message }) }
})

app.get('/emprunts/en-cours', async (req, res) => {
    try {
        const r = await axios.get(`${EMPRUNT_SERVICE}/emprunts/en-cours`)
        res.status(200).json(r.data)
    } catch (err) { res.status(500).json({ message: err.message }) }
})

app.get('/emprunts/membre/:id', async (req, res) => {
    try {
        const r = await
axios.get(`${EMPRUNT_SERVICE}/emprunts/membre/${req.params.id}`)
        res.status(200).json(r.data)
    }

```

```

    } catch (err) { res.status(500).json({ message: err.message }) }
  })

app.post('/emprunts', async (req, res) => {
  try {
    const r = await axios.post(`${EMPRUNT_SERVICE}/emprunts`, req.body)
    res.status(201).json(r.data)
  } catch (err) {
    if (err.response) return
    res.status(err.response.status).json(err.response.data)
    res.status(500).json({ message: err.message })
  }
})

app.patch('/emprunts/:id/retour', async (req, res) => {
  try {
    const r = await
    axios.patch(`${EMPRUNT_SERVICE}/emprunts/${req.params.id}/retour`)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response) return
    res.status(err.response.status).json(err.response.data)
    res.status(500).json({ message: err.message })
  }
})

app.delete('/emprunts/:id', async (req, res) => {
  try {
    const r = await axios.delete(`${EMPRUNT_SERVICE}/emprunts/${req.params.id}`)
    res.status(200).json(r.data)
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ message:
    'Emprunt non trouve' })
    res.status(500).json({ message: err.message })
  }
})

```

Demarrage du serveur :

```
app.listen(3000, () => console.log('Gateway démarre sur le port 3000'))
```

5. Etape 3 — Lancer les 4 services

Ouvrir 4 terminaux. Les 3 premiers sont exactement les memes que dans la Partie 1.

```
# Terminal 1 – inchange
cd livre-service && npm run dev

# Terminal 2 – inchange
cd membre-service && npm run dev

# Terminal 3 – inchange
cd emprunt-service && npm run dev

# Terminal 4 – nouveau
cd gateway && node index.js
```

Dans les terminaux vous devez voir :

Livre service demarre sur le port 3001
Membre service demarre sur le port 3002
Emprunt service demarre sur le port 3003
Gateway demarre sur le port 3000